强 化 学 习 实 验
实 验 报 告

实验名称：4×4 井字棋游戏

姓　　名：韩炎旸

学　　号：2312225

专　　业：智能科学与技术

人工智能学院

2026 年 3 月

一、实验要求

将井字棋游戏改为 4*4 棋盘

（1）完成代码修改，并顺利运行

（2）对比 1e4 和 1e5 训练的两个智能体对抗效果，并讨论先手和后手的胜率变化。

二、实验目的

通过强化学习中的 TD 学习算法训练两个智能体进行自我对弈，学习四子棋的最优策略，并比较不同训练轮数下的性能，以及交叉对抗的结果，观察训练是否收敛到最优策略。

三、实验原理

（1）状态表示与哈希

每个棋盘状态用一个 4×4 矩阵表示，1 代表先手棋子，-1 代表后手棋子，0 代表空位。为唯一标识状态，采用三进制哈希：从左到右遍历矩阵，每个值加 1 后累积乘 3，确保不同状态对应不同哈希值。

（2）价值函数初始化

每个状态 s 对应一个价值 V(s)，表示在当前状态下当前玩家（即将走棋的玩家）的获胜概率。终局状态根据胜负赋予确定值：当前玩家赢为 1，输为 0，平局为 0.5；非终局状态初始化为 0.5。

（3）TD 学习更新

智能体在自我对弈中记录经历的状态序列及每一步是否为贪婪动作（即未探索）。每局结束后，从倒数第二步开始反向更新，该更新方式属于 TD(0)算法，利用后继状态的价值修正当前状态估计。

（4）动作选择策略

采用ε-greedy 策略平衡探索与利用：以概率 ε 随机选择合法落子位置；否则选择使后继状态价值最大的动作（若有多个最大值则随机选取）。训练时ε=0.01，测试时ε=0 以完全利用所学策略。

四、代码修改部分

```python
def get_all_states_impl(current_state, current_symbol, all_states):
    for i in range(BOARD_ROWS):
        for j in range(BOARD_COLS):
            if current_state.data[i][j] == 0:
                new_state = current_state.next_state(i, j, current_symbol)
                new_hash = new_state.hash()
                if new_hash not in all_states:
                    is_end = new_state.is_end()
                    all_states[new_hash] = (new_state, is_end)
                    if len(all_states) % 100000 == 0:
                        print(f"Generating states... currently found {len(all_states)} states")
                    if not is_end:
                        get_all_states_impl(new_state, -current_symbol, all_states)


def get_all_states():
    current_symbol = 1
    current_state = State()
    all_states = dict()
    all_states[current_state.hash()] = (current_state, current_state.is_end())
    get_all_states_impl(current_state, current_symbol, all_states)
    return all_states


# all possible board configurations
all_states = get_all_states()
```

仍然沿用了 3*3 井字棋的状态获取函数，但是从 3*3 变为了 4*4，相对于 3*3 的棋盘来说，4*4 有了更多的状态情况，大约 970 万种，产生了组合爆炸，给内存和运算带来了压力。

```
def save_policy(self, suffix=""):
    with open('policy_%s%s.bin' % ('first' if self.symbol == 1 else 'second', suffix), 'wb
        pickle.dump(self.estimations, f)

def load_policy(self, suffix=""):
    with open('policy_%s%s.bin' % ('first' if self.symbol == 1 else 'second', suffix), 'rb
        self.estimations = pickle.load(f)
```

在 Play 类中加入了策略保存函数，方便了同时训练多个模型，从而更有利于进行组内对抗测试和组间对抗测试。

```
if __name__ == '__main__':
    epochs_list = [int(1e4), int(1e5), int(1e6)]
    p1_rates_list = []
    p2_rates_list = []

    # 分别进行训练和自身轮次模型的对抗
    for epochs in epochs_list:
        print(f"\n--- Training for {epochs} epochs ---")
        suffix = f"_{epochs}"
        train(epochs, print_every_n=epochs//10, suffix=suffix)
        print(f"--- Competing models of {epochs} epochs ---")
        r1, r2 = compete(int(1e3), p1_suffix=suffix, p2_suffix=suffix)
        p1_rates_list.append(r1)
        p2_rates_list.append(r2)

    print("\n--- Summary ---")
    for i in range(3):
        print('%d epoch, 1000 turns, player 1 win %.02f, player 2 win %.02f' % (epochs_list[i],

    print("\n--- Cross-epoch Competitions ---")
    print("1e4 First vs 1e5 Second:")
    compete(int(1e3), p1_suffix=f"_{int(1e4)}", p2_suffix=f"_{int(1e5)}")

    print("1e5 First vs 1e4 Second:")
    compete(int(1e3), p1_suffix=f"_{int(1e5)}", p2_suffix=f"_{int(1e4)}")
```

主函数种加入了信息打印和对抗设置，实现了 1e4、1e5、1e6 模型的训练，以及其组内对抗和组间对抗。

五、实验结果

（1）组内对抗结果

Epoch / Winner	1e4	1e5	1e6
Player 1	0.41	0.57	0.0
Player 2	0.39	0.21	0.0
Tie	0.2	0.22	1.0

（2）交叉对抗

Epoch / Winner	1e4 First vs 1e5 Second	1e5 First vs 1e4 Second
Player 1	0.89	0.93
Player 2	0.06	0.03
Tie	0.05	0.04

表格展示了不同 epoch 训练的模型组内对抗和组间对抗，Player 1(先手)赢、Player 2(后手)赢、平局的概率，对抗 turns 为 1000。

六、分析与总结

1. 训练收敛性分析

（1）**100 万轮同模型对抗**：先手和后手胜率均为 0，全部为平局。这表明经过足够多的训练，两个智能体均收敛到接近最优策略，实现了理论预期——双方最优玩法下必然平局。该结果验证了 TD 学习算法在完全信息博弈中的有效性，能够通过自我对弈逐步逼近纳什均衡。

（2）**10 万轮同模型对抗**：先手胜率（0.57）远高于后手（0.21），平局率仅 0.22。说明此时策略尚未收敛，后手存在明显缺陷。可能原因：

学习不对称性：先手拥有主动权，其状态空间相对较小（先手走法较少），而后手需要应对更多局面，相同训练次数下后手学习更慢。

探索策略影响：ε=0.01 的探索导致后手在早期可能接触到不利局面，而价值更新仅针对贪婪动作，可能使后手策略偏向保守或错误估计。

初始化偏差：所有非终局状态初始化为 0.5，但实际后手在面对先手优势时，许多状态的真实价值应低于 0.5，需要大量样本修正。

（3）**1 万轮同模型对抗**：先手和后手胜率接近（0.41 vs 0.39），平局率 0.20。此时训练不足，策略随机性较强，尚未形成明显优劣。随着训练增加，先手优势逐渐显现（10 万轮），但随后又因后手策略改善而趋近平局（100 万轮）。

2. 交叉对抗揭示的策略强度

（1）**训练时长主导性能**：无论是作为先手还是后手，训练时间更长的模型（10 万轮）均大幅压制训练时间短的模型（1 万轮）。例如，10 万先手对 1 万后手胜率高达 0.93，而 1 万先手对 10 万后手胜率仅 0.89（即 10 万后手仍有 6%胜率）。这表明训练轮次越多，策略越强，且优势跨越先后手。

（2）**先手优势的体现**：10 万先手对 1 万后手胜率（0.93）略高于 1 万先手对 10 万后手胜率（0.89），说明当强模型为先手时优势更大。即使强模型为后手，也能有效防守并抓住弱模型的失误取胜（6%胜率+5%平局），体现了其防守能力。

（3）**弱模型的局限性**：1 万模型作为先手时，面对 10 万后手仍能取得 89%胜率，说明其进攻策略已初步成型，但防守能力（作为后手时仅 3%胜率）严重不足，导致整体策略不稳定。